

LAB 01:

The screenshot displays two instances of the pgAdmin 4 interface, each showing a SQL query being executed. The left instance shows a query that updates the rental_rate to 4.99 for film_id 1. The right instance shows a query that updates the rental_rate to 5.99 for film_id 1. Both queries are committed, and the results are displayed in the Data Output pane.

Left Session Query:

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 4.99
5 WHERE film_id = 1;
6
7 COMMIT;
8
9 SELECT film_id, title, rental_rate
10 FROM film
11 WHERE film_id = 1;
```

Left Session Data Output:

film_id	title	rental_rate
1	Academy Dinosaur	4.99

Right Session Query:

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 5.99
5 WHERE film_id = 1;
6
7 COMMIT;
8
9 SELECT film_id, title, rental_rate
10 FROM film
11 WHERE film_id = 1;
```

Right Session Data Output:

film_id	title	rental_rate
1	Academy Dinosaur	5.99

Session B was blocked because Session A held an exclusive lock on the same row. After Session A committed, the lock was released and Session B continued. This shows that row-level locking prevents concurrent writes to the same record.

LAB 02:

The image displays two screenshots of the pgAdmin 4 interface, illustrating a database transaction scenario.

Top Screenshot: The left pane shows a query window for 'dvdrental/postgres@PostgreSQL 18'. The query is:

```
1 BEGIN;
2
3 SELECT rental_rate
4 FROM film
5 WHERE film_id = 10;
```

The 'Data Output' tab shows the result:

rental_rate
4.99

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.084'.

The right pane shows a query window for 'dvdrental/postgres@PostgreSQL 18'. The query is:

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 0.99
5 WHERE film_id = 10;
6
7 COMMIT;
```

The 'Data Output' tab shows the result:

```
COMMIT
Query returned successfully in 70 msec.
```

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.070'.

Bottom Screenshot: The left pane shows a query window for 'dvdrental/postgres@PostgreSQL 18'. The query is:

```
1 SELECT rental_rate
2 FROM film
3 WHERE film_id = 10;
4
5 COMMIT;
```

The 'Data Output' tab shows the result:

```
COMMIT
Query returned successfully in 72 msec.
```

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.072'.

The right pane shows a query window for 'dvdrental/postgres@PostgreSQL 18'. The query is:

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 0.99
5 WHERE film_id = 10;
6
7 COMMIT;
```

The 'Data Output' tab shows the result:

```
COMMIT
Query returned successfully in 73 msec.
```

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.073'.

Under the default Read Committed isolation level, Session A first reads `rental_rate = 4.99`. While Session A is still active, Session B updates the value to `0.99` and commits. When Session A reads the same row again, it sees `0.99`. This demonstrates a **non-repeatable read**, because the same transaction reads the same row twice but obtains different values.

LAB 03:

pgAdmin 4

File Object Tools Edit View Window Help

ndencies X Dependents X Processes X dvdrental/postgres@PostgreSQL 18* X

Servers

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```
1 UPDATE film
2 SET rental_rate = 3.99
3 WHERE film_id = 200;
```

Data Output Messages Notifications

Total rows: Waiting for the query to complete... 00:00:09.945 CRLF Ln 3, Col 21

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 2.99
5 WHERE film_id = 200;
```

Data Output Messages Notifications

WARNING: there is already a transaction in progress
UPDATE 1

Query returned successfully in 78 msec.

Total rows: Query complete 00:00:00.078 CRLF Ln 5, Col 21

pgAdmin 4

File Object Tools Edit View Window Help

ndencies X Dependents X Processes X dvdrental/postgres@PostgreSQL 18* X

Servers

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```
1 UPDATE film
2 SET rental_rate = 3.99
3 WHERE film_id = 200;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 38 secs 374 msec.

Total rows: Query complete 00:00:38.374 CRLF Ln 3, Col 21

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad X

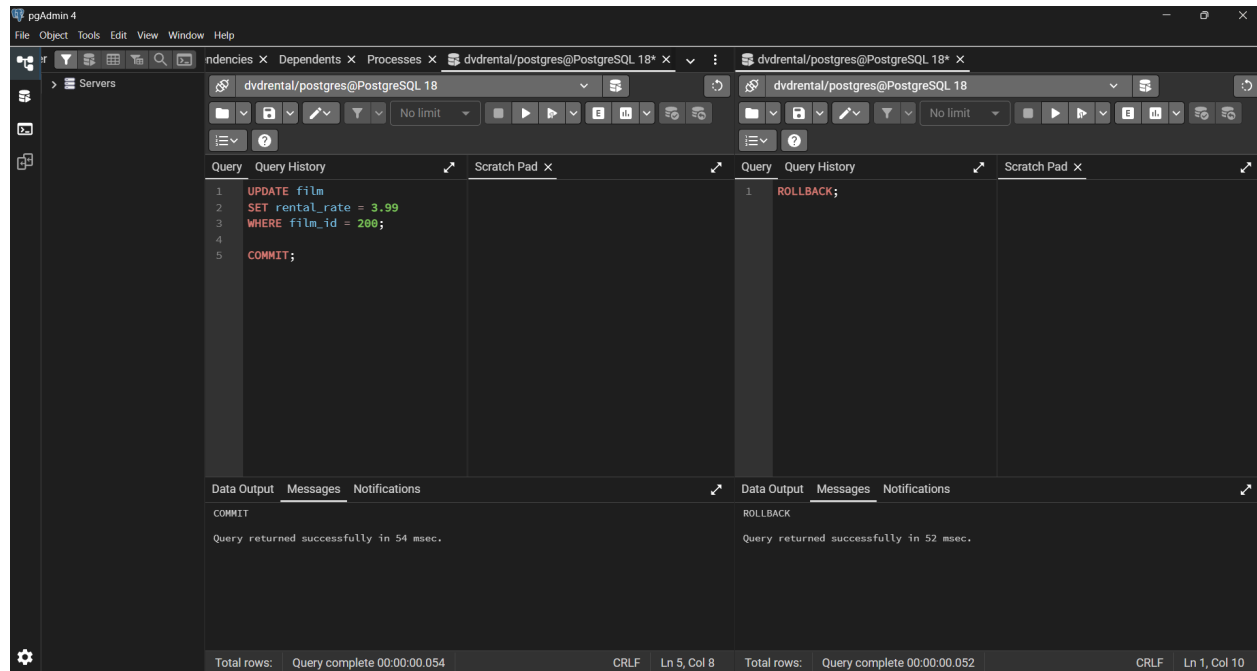
```
1 UPDATE film
2 SET rental_rate = 3.99
3 WHERE film_id = 100;
```

Data Output Messages Notifications

ERROR: deadlock detected
Process 59996 waits for ShareLock on transaction 979; blocked by process 35828.
Process 35828 waits for ShareLock on transaction 980; blocked by process 59996.

SQL state: 40P01
Detail: Process 59996 waits for ShareLock on transaction 979; blocked by process 35828.
Process 35828 waits for ShareLock on transaction 980; blocked by process 59996.
Hint: See server log for query details.
Context: white locking tuple (53,17) in relation "film"

Total rows: Query complete 00:00:01.048 CRLF Ln 3, Col 21



A deadlock occurred because Session A locked `film_id = 100` and waited for `film_id = 200`, while Session B locked `film_id = 200` and waited for `film_id = 100`. This created a circular wait. PostgreSQL detected the deadlock and rolled back one transaction so that the other transaction could continue.

LAB 04:

pgAdmin 4

File Object Tools Edit View Window Help

ndencies × Dependents × Processes × dvdrental/postgres@PostgreSQL 18* × dvdrental/postgres@PostgreSQL 18*

Servers

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad

```
1 BEGIN;
2
3 SELECT *
4 FROM inventory
5 WHERE inventory_id = 10
6 FOR UPDATE;
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1 Page No: 1

inventory_id	film_id	store_id	last_update
[PK] integer	smallint	smallint	timestamp without time zone
1	10	2	2006-02-15 10:09:17

Total rows: 1 Query complete 00:00:00.087 CRLF Ln 6, Col 12

pgAdmin 4

File Object Tools Edit View Window Help

ndencies × Dependents × Processes × dvdrental/postgres@PostgreSQL 18* × dvdrental/postgres@PostgreSQL 18*

Servers

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad

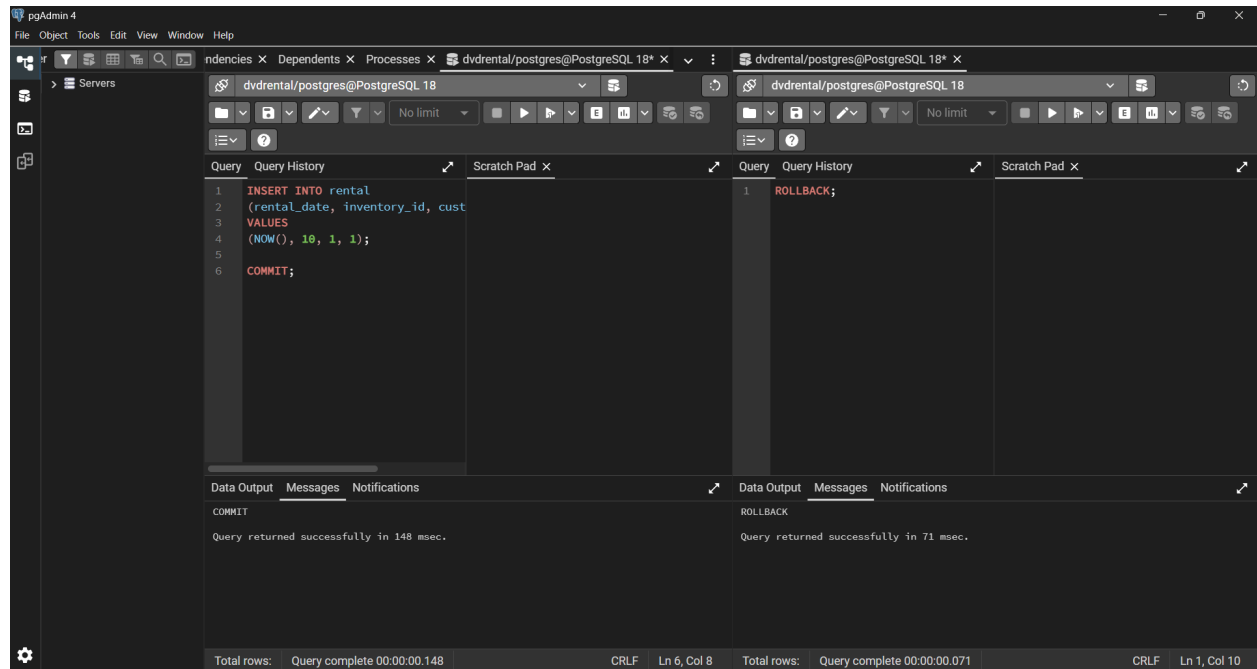
```
1 INSERT INTO rental
2 (rental_date, inventory_id, cust
3 VALUES
4 (NOW(), 10, 1, 1);
5
6 COMMIT;
```

Data Output Messages Notifications

COMMIT

Query returned successfully in 148 msec.

Total rows: 1 Query complete 00:00:00.148 CRLF Ln 6, Col 8



Session A locked `inventory_id = 10` using `SELECT ... FOR UPDATE`. When Session B attempted to lock the same row, it was blocked until Session A completed its transaction. After Session A inserted the rental and committed, Session B continued and could detect that the DVD had already been rented. Therefore, the second transaction was rolled back, preventing a double-booking race condition.